ENGENHARIA DE SOFTWARE COM AGENTES

Matt Pocock

Rigor de Engenharia com Agentes

Alinhar antes de construir, testar antes de implementar e corrigir o ambiente em vez do passado

OS SEIS NÓS DO LOOP matt-pocock

1. **Alinhar Domínio**
2. **Especificar**
3. **Planejar Tickets**
4. **Construir com TDD**
5. **Revisar em Dois Eixos**
6. **Retroalimentar o Ambiente**

Heverton & Ecossistema LegoLoop

07/11/2026 · LegoLoop Foundation

SOBRE ESTA OBRA

Este livro ensina a operar o método de Matt Pocock (upstream sob licença MIT) dentro do ecossistema LegoLoop. O método substitui a geração cega de código por um ciclo disciplinado: entrevistar a ideia até não restarem premissas ocultas, transformar o consenso em especificação, fatiar a especificação em tickets com dependências explícitas e implementar cada ticket com testes primeiro, em contexto limpo.

A Parte I apresenta os cinco pilares do método. A Parte II descreve o loop mestre e seus quatro gates. A Parte III detalha os seis sub-loops e as habilidades de cada um. A Parte IV desce ao nível de implementação: roteamento de modelos, higiene de contexto e o arquivo loops/matt-pocock.loop.json. A Parte V traz receitas e a Parte VI reúne glossário, relatório de conformidade e referência das habilidades.

Fonte primária: docs/manuals/2026-11-01_MANUAL_MATT_POCOCK_SKILLS_LOOP.md, complementada pelas habilidades upstream tdd, code-review e diagnosing-bugs.

Documento gerado deterministicamente a partir das fontes citadas no próprio texto, via pandoc + typst. Toda afirmação técnica desta obra é rastreável a um arquivo nomeado no capítulo correspondente.

Data da compilação: 07/11/2026 · LegoLoop Foundation

Sumário

ANOTAÇÕES

Como ler este livro 2
A quem este livro se destina 2
A estrutura da obra 2
As convenções da obra 2
PARTE I — Macro: O Fim da Geração Cega 3
1. O problema: código que chega antes da decisão 3
2. Os cinco pilares do método 4
3. O que o leitor leva desta parte 5
PARTE II — Meso: O Loop Mestre de Rigor 6
4. Da ideia à produção em cinco estágios 6
5. Os gates e o que cada um exige 7
6. Por que a ordem importa 7
PARTE III — Meso: Os Seis Sub-Loops 9
7. Visão geral 9
8. Sub-loop 1: Concepção, Grilling e Domínio 9
9. Sub-loop 2: Exploração Empírica e Pesquisa 10
10. Sub-loop 3: Especificação e Grafo 10
11. Sub-loop 4: Construção Cirúrgica e TDD 11
12. Sub-loop 5: Diagnóstico, Arquitetura e Triagem 12
13. Sub-loop 6: Auditoria, Entrega e Ambiente 12
PARTE IV — Micro: Roteamento, Contexto e Integração 14
14. Roteamento cognitivo de modelos 14
15. Higiene de contexto e fronteiras de fase 14
16. Integração no LegoLoop 15
PARTE V — Prática: Receitas de Uso 17
17. Receita: uma funcionalidade nova, do zero 17
18. Receita: um bug difícil 17
19. Receita: uma dúvida que só código responde 18
20. Receita: revisar o trabalho de outro agente 18
PARTE VI — Apêndices 20
Apêndice A: Glossário 20
Apêndice B: Relatório de conformidade 20
Apêndice C: Referência rápida das habilidades 21

Como ler este livro

ANOTAÇÕES

A quem este livro se destina

1. **Quem lidera times que usam agentes.** Precisa entender por que "deixar a IA programar" produz código que ninguém consegue manter, e onde o humano precisa continuar decidindo. A Parte I responde a isso.
2. **Quem desenvolve com agentes no dia a dia.** Precisa saber qual habilidade chamar, quando limpar o contexto e o que cada gate exige. As Partes II, III e V são o núcleo para esse perfil.
3. **Quem mantém a plataforma LegoLoop.** Precisa saber onde o pipeline está declarado, quais artefatos cada nó exige e onde o manual e o código divergem. A Parte IV e o Apêndice B são escritos para esse leitor.

A estrutura da obra

Parte	Nível	Conteúdo
I	Macro	O problema da geração cega e os cinco pilares do método
II	Meso	O loop mestre, da ideia à produção, com quatro gates
III	Meso	Os seis sub-loops e suas habilidades
IV	Micro	Roteamento de modelos, higiene de contexto e integração
V	Prática	Receitas para situações frequentes
VI	Apêndices	Glossário, relatório de conformidade e referência rápida

As convenções da obra

Nomes de habilidades e caminhos de arquivo aparecem em fonte mono-espaçada, como `grill-with-docs` ou `loops/matt-pocock.loop.json`. Exemplos ilustrativos são marcados como tal no texto. Toda afirmação sobre o LegoLoop é rastreável a um arquivo citado; quando manual e código discordam, o livro segue o código e registra a diferença no Apêndice B.

PARTE I — Macro: O Fim da Geração Cega

ANOTAÇÕES

1. O problema: código que chega antes da decisão

Um agente de IA consegue escrever centenas de linhas em segundos. Esse é o problema. Quando o pedido é "adicione um sistema de assinaturas", o agente não para para perguntar se uma assinatura pertence a um usuário ou a uma conta, se o cancelamento vale no fim do ciclo ou imediatamente, ou o que acontece com um pagamento recusado. Ele escolhe uma resposta plausível para cada pergunta, sem avisar, e escreve o código.

O resultado parece pronto. Os testes passam, porque foram escritos pelo mesmo agente, a partir das mesmas suposições. Semanas depois, alguém descobre que "conta" significa coisas diferentes em três módulos e que a regra de cancelamento contradiz o contrato comercial.

O método de Matt Pocock parte de um diagnóstico simples: o gargalo do desenvolvimento com agentes não é escrever código, é decidir o que escrever. Por isso o método gasta esforço deliberado antes da primeira linha de produção e mantém o humano no controle das decisões que são difíceis de desfazer.

Três sintomas da geração cega

- **Premissas silenciosas.** O agente preencheu lacunas do pedido com suposições que ninguém aprovou.
- **Testes que concordam com o código por construção.** O teste recalcula o resultado do mesmo jeito que a implementação e, por isso, nunca discorda dela.
- **A sessão que apodrece.** Depois de horas de conversa, o agente começa a esquecer decisões do início, a misturar tentativas descartadas com a versão atual e a errar coisas que acertava antes.

2. Os cinco pilares do método

ANOTAÇÕES

Pilar 1: alinhar antes de construir

Em inglês, *grill before build*. Nenhuma linha de código de produção é gerada antes de a ideia passar por uma entrevista implacável que exponha premissas ocultas, casos de borda e ambiguidades de domínio. A entrevista não é uma formalidade: ela termina com decisões registradas em arquivo.

Pilar 2: o desenvolvedor no controle

O agente opera em execuções curtas, de três a cinco passos, com gates de parada obrigatória. Ações críticas, como push, abertura de pull request, deploy e deleção, exigem autorização humana expressa.

> **POR QUE EXECUÇÕES CURTAS**
>
> Um agente que executa vinte passos sem parar acumula erros que se apoiam uns nos outros. Em três a cinco passos, o desvio é pequeno o bastante para ser notado e barato o bastante para ser desfeito.

Pilar 3: módulos profundos

O pilar vem de John Ousterhout e do livro *A Philosophy of Software Design*. Um módulo profundo tem uma interface estreita e simples que esconde uma implementação rica. O contrário, o módulo raso, expõe quase tanta complexidade na interface quanto contém por dentro.

O método usa a palavra *seam*, costura, para a fronteira pública em que o comportamento é observado sem olhar para dentro. É nas costuras que os testes moram.

Pilar 4: higiene severa de contexto

Modelos de linguagem degradam o raciocínio quando a janela de contexto cresce além de uma zona de clareza que o manual estima em cerca de 150 mil tokens. O método chama a faixa saudável de *Smart Zone* e organiza o trabalho para ficar dentro dela.

ANOTAÇÕES

A regra prática é dupla: alinhamento, especificação e tickets acontecem em uma janela única e contínua, porque compartilham raciocínio; a implementação de cada ticket começa em contexto limpo, porque só precisa do ticket e do repositório.

Pilar 5: retroalimentar o ambiente

Retrospectivas não mudam o código passado. Elas mudam as regras, os linters, as checagens automáticas e o `AGENTS.md`, para que a próxima sessão não repita o mesmo erro.

> **Um erro corrigido no código volta. Um erro corrigido no ambiente não volta.**

3. O que o leitor leva desta parte

O método não é um conjunto de prompts melhores. É uma divisão de trabalho: o humano decide o que é difícil de reverter, o agente executa fatias pequenas e verificáveis, e cada falha vira uma regra automática. A tabela abaixo antecipa onde cada pilar se materializa no loop.

Pilar	Onde aparece no loop
Alinhar antes de construir	Sub-loop 1 e gate G1
Desenvolvedor no controle	Gates G1 a G4 e ações críticas autorizadas
Módulos profundos	`codebase-design` e testes em costuras
Higiene de contexto	Janela única até os tickets, `/clear` depois
Retroalimentar o ambiente	`retro` e o gate G4

PARTE II — Meso: O Loop Mestre de Rigor

ANOTAÇÕES

4. Da ideia à produção em cinco estágios

O loop mestre atravessa cinco estágios. Quatro gates binários separam os estágios principais: G1 a G4. O segundo estágio, a prova de conceito, é opcional e funciona como um desvio que volta ao primeiro.

1. Alinhamento e Domínio grill-with-docs, domain-modeling

↓ **G1: decisões aprovadas**

3. Especificação e Grafo to-spec, to-tickets

↓ **G2: tickets e bloqueios aprovados**

4. Construção TDD implement, tdd, codebase-design

↓ **G3: testes 100% verdes**

5. Fechamento e Entrega code-review, pr, retro

Depois do estágio 5, o gate G4, review aprovado, marca a entrega como concluída.

O desvio empírico: estágio 2

Às vezes uma pergunta do alinhamento não se resolve conversando. "Esse modelo de estado aguenta edição concorrente?" só se responde com código rodando. Nesse caso o fluxo sai do estágio 1, abre uma sessão isolada com `handoff`, constrói um protótipo descartável com `prototype` e volta com a resposta por outro `handoff`. O protótipo não entra na árvore principal.

> **PROTÓTIPO NÃO É RASCUNHO DE PRODUÇÃO**
>
> O protótipo responde uma pergunta e é jogado fora. Se o código do protótipo for aproveitado na implementação, ele entra sem spec, sem tickets e sem testes primeiro, que é exatamente o que o método existe para evitar.

5. Os gates e o que cada um exige

ANOTAÇÕES

Gate	Entre	Condição para passar
G1	Alinhamento e Especificação	Ambiguidades de domínio sanadas e aprovadas
G2	Especificação e Construção	DAG de tickets aprovado, bloqueios resolvidos
G3	Construção e Fechamento	Testes 100% verdes, sem testes enfraquecidos
G4	Fechamento e Entrega	Review sem notas bloqueadoras, ambiente atualizado

G1: decisões aprovadas

O estágio de alinhamento termina quando o desenvolvedor aprova as decisões. Não basta o agente achar que entendeu; as decisões precisam estar escritas no `GLOSSARY.md`, em um documento de intenção ou em ADRs.

G2: tickets e bloqueios aprovados

A especificação vira um grafo acíclico de tickets. O gate exige que cada aresta de bloqueio esteja explícita: o ticket 4 depende do 2, o 5 não depende de ninguém. Um ticket com dependência implícita quebra a execução em contexto limpo, porque a sessão que o implementa não sabe o que veio antes.

G3: testes 100% verdes

O manual detalha o que conta como verde: nenhum teste enfraquecido e nenhuma supressão de compilador. Um teste que foi afrouxado para passar, ou um `@ts-ignore` colocado para calar um erro de tipo, reprovam o gate mesmo com a suíte verde.

G4: review aprovado

O review em dois eixos não pode ter notas bloqueadoras, e a retrospectiva precisa ter atualizado o ambiente. Uma entrega sem retro passa no código, mas reprova no método.

6. Por que a ordem importa

Cada estágio produz o insumo exato do seguinte. A especificação depende de um vocabulário fixado; os tickets dependem da especificação; a implementação depende de tickets autocontidos; o review depende da especificação para o segundo eixo.

Estágio	Produz	Consumido por
Alinhamento	`GLOSSARY.md`, intenção, ADRs	Especificação
Especificação	Spec e DAG de tickets	Construção
Construção	Código, testes, relatório TDD	Fechamento
Fechamento	Review, corpo do PR, `AGENTS.md`	A próxima sessão

ANOTAÇÕES

PARTE III — Meso: Os Seis Sub--Loops

ANOTAÇÕES

7. Visão geral

O manual decompõe o ecossistema de Matt Pocock em seis sub-loops, disponíveis como blocos conectáveis no Studio Canvas e em pipelines DAG.

Sub--loop	Nome	Gate de saída
1	Concepção, Grilling e Domínio	Ambiguidades sanadas e aprovadas
2	Exploração Empírica e Pesquisa	Resposta empírica devolvida via `handoff`
3	Especificação e Grafo	DAG aprovado, bloqueios resolvidos
4	Construção Cirúrgica e TDD	Testes verdes, sem testes enfraquecidos
5	Diagnóstico e Arquitetura	Falha reproduzida, corrigida e protegida
6	Auditoria, Entrega e Ambiente	Review aprovado e ambiente atualizado

8. Sub-loop 1: Concepção, Grilling e Domínio

O objetivo é fixar um vocabulário unívoco, desafiar premissas e registrar decisões arquiteturais antes de qualquer código.

- `grill-with-docs` é a entrevista socrática focada no repositório. Deixa rastro durável em `GLOSSARY.md` e em ADRs.
- `grill-me` é a variante sem estado, para alinhamento rápido sem pasta de projeto.
- `grilling` é o primitivo da entrevista: rodadas numeradas e separação clara entre o que o agente sabe e o que o usuário decide.
- `domain-modeling` audita termos sobrecarregados, como "usuário" e "conta", e documenta decisões difíceis de reverter.

Saídas: `GLOSSARY.md`, `docs/intents/YYYY-MM-DD_INTENT_<TEMA>.md` e ADRs em `docs/adr/`.

ANOTAÇÕES

FATOS DO AGENTE E DECISÕES DO USUÁRIO

A separação que o grilling impõe é a parte mais valiosa da entrevista. O agente pode afirmar fatos que leu no repositório: o campo tal existe, a tabela tal tem tal índice. Ele não pode afirmar decisões: se o cancelamento é imediato é uma escolha de produto, e só o usuário a faz.

9. Sub-loop 2: Exploração Empírica e Pesquisa

O objetivo é responder perguntas que só código executável ou fontes primárias resolvem, sem poluir a árvore principal.

- `prototype` cria um programa descartável na branch `prototype/<nome>`, para testar um modelo de estado ou a sensação de uma interface.
- `handoff` passa o contexto de forma estéril para uma nova pasta, harness ou sessão.
- `research` roda uma pesquisa profunda em segundo plano sobre documentação oficial e fontes primárias.

Saídas: relatório técnico com citações e protótipo referenciado na branch isolada.

10. Sub-loop 3: Especificação e Grafo

O objetivo é sintetizar o consenso em uma especificação formal e fatiá-la em um grafo acíclico de tarefas independentes.

- `to-spec` consolida a discussão em uma spec executável e determinística, sem entrevistar de novo.
- `to-tickets` quebra a spec em tickets verticais, os *tracer bullets*, cada um com suas arestas de bloqueio (`blocked-by`).
- `wayfinder` traça um mapa de tickets de decisão para iniciativas nebulosas ou projetos grandes começando do zero.

Saídas: `docs/specs/YYYY-MM-DD_SPEC_<TEMA>.md` e tickets em `.scratch/<feature>/issues/NN-slug.md` ou `docs/plans/`.

ANOTAÇÕES

Um ticket vertical atravessa todas as camadas necessárias para entregar um comportamento observável, da interface ao banco. O contrário, o ticket horizontal ("criar todas as tabelas"), não entrega nada testável sozinho.

11. Sub-loop 4: Construção Cirúrgica e TDD

O objetivo é implementar orientado a testes, com contexto isolado por ticket.

- `implement` consome um ticket por vez, a partir de uma janela limpa, atacando nós desbloqueados.
- `implement-spec` distribui subagentes em paralelo sobre os tickets prontos e unifica em uma branch de integração.
- `tdd` conduz o ciclo vermelho-verde por fatia de comportamento, sem mocks de produção e sem stubs vazios.
- `codebase-design` fornece o vocabulário de módulos profundos, interfaces estreitas e costuras.

Saídas: código de produção, testes e `docs/reports/TDD_REPORT.md`.

O que a habilidade tdd exige

O texto upstream da habilidade `tdd` é mais preciso do que o resumo do manual em três pontos.

1. **Testes só em costuras combinadas.** Antes de escrever qualquer teste, o agente lista as costuras que serão testadas e confirma com o usuário. Nenhum teste é escrito em costura não confirmada.
2. **Três anti-padrões nomeados.** O teste acoplado à implementação, que quebra em refatorações sem mudança de comportamento; o teste tautológico, que recalcula o esperado do mesmo jeito que o código; e o fatiamento horizontal, que escreve todos os testes antes de toda a implementação.
3. **Refatorar não faz parte do loop.** O ciclo é vermelho e verde. A refatoração pertence ao estágio de review, conduzido pela habilidade `code-review`.

ANOTAÇÕES

O TESTE TAUTOLÓGICO

expect(soma(a, b)).toBe(a + b) passa por construção: o esperado é calculado do mesmo jeito que o código calcula. O valor esperado precisa vir de uma fonte independente: um literal sabidamente correto, um exemplo resolvido à mão ou a especificação.

12. Sub-loop 5: Diagnóstico, Arquitetura e Triagem

O objetivo é resolver falhas complexas e manter a saúde arquitetural.

- `diagnosing-bugs` proíbe teorizar antes de obter um loop de feedback reproduzível: um comando que fique vermelho por causa deste bug. A correção vem acompanhada de teste de regressão.
- `improve-codebase-architecture` procura módulos rasos e oportunidades de aprofundamento.
- `triage` qualifica e enriquece issues externas antes de despachá-las para agentes.

O texto upstream de `diagnosing-bugs` chama a construção do loop de feedback de "a habilidade em si" e lista formas de obtê-lo, em ordem: teste falhando, script HTTP contra o servidor local, chamada de CLI comparando a saída com um snapshot, script de navegador headless e reprodução de um trace capturado. A mesma habilidade exige ocultar todo segredo como `<REDACTED>` antes de mostrar comandos e saídas.

13. Sub-loop 6: Auditoria, Entrega e Ambiente

O objetivo é revisar a conformidade, redigir o PR e blindar o ambiente.

- `code-review` audita em dois eixos ortogonais: *Standards*, as regras documentadas do repositório, e *Spec*, a fidelidade à tarefa de origem.
- `pr` estrutura o pull request com a menor evidência visual necessária, antes e depois, e classifica o risco como porta de mão única ou de mão dupla.

ANOTAÇÕES

- `retro` converte falhas mecânicas em checagens automáticas e divergências em regras de linter ou do `AGENTS.md`.
- `wizard` gera scripts interativos para etapas que exigem presença humana, como tokens e senhas.
- `wait-what` re-explica um conceito sem jargão quando o usuário se perde.

Saídas: `docs/reports/CODE_REVIEW.md`, corpo do PR e atualizações no `AGENTS.md`.

O upstream de `code-review` acrescenta um detalhe de mecânica: os dois eixos rodam em subagentes paralelos, para que um não contamine o contexto do outro, sobre o diff de três pontos entre um ponto fixo informado pelo usuário e o `HEAD`.

PARTE IV — Micro: Roteamento, Contexto e Integração

ANOTAÇÕES

14. Roteamento cognitivo de modelos

O manual recomenda um modelo primário e um alternativo local para cada sub-loop. A lógica é gastar raciocínio caro onde há ambiguidade e geração rápida onde há execução mecânica.

Sub-loop	Complexidade	Primário	Alternativo local
1. Concepção	Alta (socrática)	`sonnet`	`haiku`
2. Exploração	Média	`code-fast`	`qwen2.5-coder:7b`
3. Especificação	Alta (arquitetura)	`code-pro`	`llama3.1:8b`
4. Construção TDD	Média a alta	`sonnet`	`qwen2.5-coder:7b`
5. Diagnóstico	Alta	`sonnet`	`llama3.1:8b`
6. Review e Retro	Média	`code-fast`	`haiku`

O raciocínio socrático do sub-loop 1 pede um modelo capaz de desafiar premissas. Protótipos descartáveis do sub-loop 2 pedem velocidade, não profundidade. O review do sub-loop 6 é uma verificação binária de regras, em que um modelo rápido basta.

15. Higiene de contexto e fronteiras de fase

As duas regras fixas

1. **Janela unificada de alinhamento.** Grilling, `to-spec` e `to-tickets` ficam na mesma sessão, porque compartilham raciocínio.
2. **Contexto limpo por ticket.** Cada `implement` começa em sessão limpa, consumindo apenas o arquivo do ticket e o repositório.

A decisão na fronteira de fase

Situação na fronteira	Ação
Fase curta, contexto abaixo de 100 mil tokens	Continuar
Fase terminou, próximo ticket autocontido	`/clear`
Alinhamento perto do limite, antes dos tickets	`/compact`
Bifurcação lateral ou protótipo	`/handoff`

ANOTAÇÕES

COMPACTAR NÃO É LIMPAR

/compact resume a conversa e mantém o resumo; serve quando o raciocínio ainda é necessário, como no fim do alinhamento. /clear descarta tudo; serve quando o próximo passo é autocontido, como um ticket. Usar /compact antes de cada ticket carrega para a implementação resíduos de decisões que já estão no ticket.

16. Integração no LegoLoop

O arquivo do loop

O arquivo `loops/matt-pocock.loop.json` segue o esquema `dag.v1`, chama-se "Pipeline de Rigor Matt Pocock (6 Sub-Loops)", versão 1.0.0, e tem gatilho manual. Os seis nós apontam para a peça `skills/aidd-pocock` e formam uma cadeia linear. Cada nó declara o artefato que precisa existir para ser concluído.

Nó	Artefato exigido pelo gate	Modelo sugerido
`subloop-1-alinhar-dominio`	`GLOSSARY.md`	sonnet, ou haiku
`subloop-2-especificar`	`docs/specs/SPEC.md`	não declarado
`subloop-3-planejar-tickets`	`docs/plans/PLAN.md`	não declarado
`subloop-4-construir-tdd`	`docs/reports/TDD_REPORT.md`	sonnet, ou qwen2.5-coder
`subloop-5-review-auditar`	`docs/reports/CODE_REVIEW.md`	não declarado
`subloop-6-retro-ambiente`	`docs/reports/RETRO.md`	não declarado

O gate de cada nó é do tipo `file_exists(...)`. Ele confirma que o artefato foi produzido, não que está correto. O julgamento continua com os gates humanos G1 a G4.

O template do Studio

Em `internal/studio/templates.go`, o template "Pipeline Matt Pocock (6 Sub--Loops)" aparece no menu Templates do Studio Canvas com o ícone `git`. Ele carrega o grafo com nós de alinhamento, especificação, tickets, TDD, review e retroalimentação.

Comandos de verificação

```
legoloop validate loops/matt-pocock.loop.json   # contrato, gates e ciclos
```

```
legoloop run loops/matt-pocock.loop.json        # execução pelo motor
DAG
legoloop studio                                 # Studio em
127.0.0.1:7878
```

Em 07/11/2026, o `validate` respondeu `6 nós válidos, sem ciclos`, em seis camadas de um nó cada, de `subloop-1-alinhar-dominio` a `subloop-6-retro-ambiente`.

ANOTAÇÕES

PARTE V — Prática: Receitas de Uso

ANOTAÇÕES

As receitas combinam sub-loops para situações frequentes. Os nomes de funcionalidades e os números são ilustrativos.

17. Receita: uma funcionalidade nova, do zero

Quando usar. Há uma ideia, por exemplo "permitir que clientes pausem a assinatura", e nenhuma linha de código.

1. Rode `grill-with-docs`. Responda as rodadas até que "pausar" tenha um significado único: quanto tempo, quem paga, o que acontece com o ciclo.
2. Confirme que as decisões estão em `GLOSSARY.md` e, se alguma for difícil de reverter, em uma ADR. Este é o G1.
3. Na mesma sessão, rode `to-spec` e depois `to-tickets`.
4. Revise o DAG: cada ticket precisa dizer de quem depende. Este é o G2.
5. Limpe o contexto. Para cada ticket desbloqueado, abra uma sessão limpa e rode `implement` com `tdd`.
6. Com a suíte verde e sem testes enfraquecidos (G3), rode `code-review`, `pr` e `retro`.

Erro comum. Pular o passo 5 e implementar os tickets na mesma sessão do alinhamento. A sessão já carrega toda a conversa, e o agente passa a implementar a partir dela, e não do ticket.

18. Receita: um bug difícil

Quando usar. "O total do carrinho às vezes sai errado." Ninguém sabe reproduzir.

1. Rode `diagnosing-bugs` e não aceite hipóteses antes do passo 2.

ANOTAÇÕES

2. Construa um loop de feedback: um comando que fique vermelho por causa deste bug. Tente, nesta ordem, teste, script HTTP, CLI com snapshot, navegador headless e reprodução de trace.
3. Com o vermelho em mãos, investigue.
4. Corrija com o menor patch possível e veja o mesmo comando ficar verde.
5. Transforme o comando em teste de regressão permanente.
6. Rode `retro`: se o bug veio de uma classe de erro detectável, converta-a em checagem automática.

> **SEM LOOP, SEM DIAGNÓSTICO**
>
> Se nenhuma das formas de construir o loop funcionar, o passo seguinte é dizer isso ao usuário e pedir mais informação, não teorizar. Teorizar sem um sinal vermelho produz correções que parecem plausíveis e não se pode provar que funcionaram.

19. Receita: uma dúvida que só código responde

Quando usar. No meio do alinhamento surge a pergunta "uma fila em memória aguenta o pico de pedidos?".

1. Interrompa o grilling e rode `handoff` para uma sessão isolada.
2. Na sessão nova, rode `prototype` na branch `prototype/fila-pico`.
3. Meça, anote o resultado e descarte o código.
4. Volte com `handoff`, levando só a resposta e a evidência.
5. Retome o grilling com a decisão informada.

20. Receita: revisar o trabalho de outro agente

Quando usar. Um agente entregou uma branch dizendo que está pronta.

1. Fixe o ponto de comparação, por exemplo `main`, e confirme que ele existe e que o diff não está vazio.

ANOTAÇÕES

2. Localize a spec de origem: referência de issue nos commits, caminho passado como argumento ou arquivo em `docs/specs/`.
3. Rode `code-review`. Os dois eixos, Standards e Spec, rodam em subagentes separados.
4. Trate cada nota bloqueadora antes do PR. Notas não bloqueadoras viram tickets.
5. Rode `retro` sobre o que o review encontrou que uma checagem automática poderia ter pego.

O passo 1 existe porque um ponto de comparação inválido deve falhar logo, e não dentro de dois subagentes paralelos.

PARTE VI — Apêndices

ANOTAÇÕES

Apêndice A: Glossário

ADR. Registro de decisão arquitetural; documenta uma escolha difícil de reverter e o motivo.

Costura (seam). Fronteira pública em que o comportamento é observado sem olhar para dentro do módulo. Os testes moram nas costuras.

DAG. Grafo acíclico direcionado; aqui, o grafo de tickets com suas arestas de bloqueio.

Grilling. Entrevista socrática em rodadas que separa fatos do agente de decisões do usuário.

Handoff. Passagem estéril de contexto para outra sessão, pasta ou harness.

Módulo profundo. Módulo com interface estreita que esconde implementação rica.

Porta de mão única. Mudança difícil ou impossível de reverter; exige mais evidência no PR.

Smart Zone. Faixa da janela de contexto em que o modelo raciocina com clareza; o manual a estima em até cerca de 150 mil tokens.

Teste tautológico. Teste cujo valor esperado é calculado do mesmo jeito que o código, e que por isso nunca falha.

Tracer bullet. Ticket vertical que entrega um comportamento observável atravessando todas as camadas.

Apêndice B: Relatório de conformidade

Comparação entre o manual de 01/11/2026 e o repositório em 07/11/2026.

ANOTAÇÕES

Ponto verificado	Manual	Código ou upstream	Situação
Arquivo do loop	`matt-pocock.loop.json`	Existe em `loops/`	Conforme
Validação do DAG	Não citada	6 nós, sem ciclos	Conforme
Sub-loop 2, Exploração	Sub-loop próprio	Sem nó no JSON	Diverge
Sub-loop 5, Diagnóstico	Sub-loop próprio	Sem nó no JSON	Diverge
Spec e tickets	Um sub-loop	Dois nós separados	Diverge
Modelos dos nós 1 e 4	sonnet/haiku, sonnet/qwen	Iguais em `routing`	Conforme
Modelos dos nós 2,3,5,6	Declarados na matriz	Ausentes em `routing`	Diverge
Template do Studio	Ícone `git`	Ícone `git`	Conforme
Ciclo do TDD	Red-Green-Refactor	Upstream: só vermelho e verde	Diverge

Leitura do relatório. O JSON reorganiza os seis sub-loops do manual: Exploração e Diagnóstico ficam fora da esteira declarada, por serem desvios sob demanda, e Especificação vira dois nós, spec e tickets. O upstream da habilidade `tdd` tira a refatoração do ciclo e a coloca no review; este livro segue o upstream.

Apêndice C: Referência rápida das habilidades

As habilidades estão preservadas na cópia local da skill `aidd-pocock`, em `upstream/skills/`: 20 em `engineering/` e 7 em `productivity/`.

Habilidade	Pasta	Etapa	Função resumida
`grill-with-docs`	engineering	1	Entrevista com rastro em arquivo
`grill-me`	productivity	1	Entrevista rápida sem estado
`grilling`	productivity	1	Primitivo da entrevista
`domain-modeling`	engineering	1	Termos sobrecarregados e ADRs
`prototype`	engineering	2	Código descartável em branch isolada
`handoff`	productivity	2	Passagem estéril de contexto
`research`	engineering	2	Pesquisa em fontes primárias
`to-spec`	engineering	3	Spec a partir do consenso
`to-tickets`	engineering	3	Tickets verticais com bloqueios
`wayfinder`	engineering	3	Mapa de decisões para iniciativas vagas
`implement`	engineering	4	Um ticket por sessão limpa
`implement-spec`	engineering	4	Tickets em paralelo com integração
`tdd`	engineering	4	Vermelho e verde em costuras combinadas
`codebase-design`	engineering	4	Vocabulário de módulos profundos
`diagnosing-bugs`	engineering	5	Loop de feedback antes de teorizar
`improve-codebase-architecture`	engineering	5	Caça a módulos rasos
`triage`	engineering	5	Qualifica issues externas
`code-review`	engineering	6	Review em Standards e Spec
`pr`	engineering	6	PR com evidência mínima e risco
`retro`	engineering	6	Falhas viram regras do ambiente
`wizard`	engineering	6	Scripts para etapas humanas
`wait-what`	productivity	6	Re-explicação sem jargão

Fora dos sub-loops ficam `ask-matt` e `setup-matt-pocock-skills`, em `engineering/`, e `teach`, `to-questionnaire` e `writing-for-agents`, em `productivity/`.